ASSIGNMENT

Course: Full Stack Web Development
Course Code: 23CM4121
Assignment No.: 1
Student Name: G.kavya
Roll No.: A24126552082
Date: 30-08-2026

TITLE

Implement Different Types of Inheritance in JavaScript

AIM

To understand and implement different types of inheritance in JavaScript using classes, the extends keyword, and mixins.

OBJECTIVES

- Implement single inheritance using JavaScript classes.
- Demonstrate multilevel inheritance through a chain of classes.
- Implement hierarchical inheritance with a common parent class.
- Demonstrate multiple inheritance behavior using mixins and Object.assign().
- Observe the output produced by each inheritance example.

CONCEPTS COVERED

- Classes and Objects
- extends keyword
- super() constructor call
- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance using Mixins
- Object.assign()
- Method Reuse and Method Extension

TASK DESCRIPTION

Part A: Single Inheritance – Create an Animal base class and a Dog derived class. Demonstrate inherited eat() and child-specific bark() methods.

Part B: Multilevel Inheritance – Create Vehicle, Bike, and Car classes in a three-level inheritance chain and invoke methods from all levels.

Part C: Hierarchical Inheritance – Create a User base class and derive Admin and Customer classes, each providing its own specialized operation.

Part D: Multiple Inheritance – Use mixins for Hall and BedRoom behavior, combine them with the Area class using Object.assign(), and inherit house() from Building.

ALGORITHM / PROCEDURE

1. Define the Animal class with a name property and an eat() method.
2. Create Dog using extends Animal and add the bark() method.
3. Create a Dog object and call both inherited and child methods.
4. Define Vehicle, Bike, and Car to demonstrate multilevel inheritance.
5. Create a Car object and invoke move(), wheels(), and windows().
6. Define User as the parent class and Admin and Customer as child classes.
7. Create Admin and Customer objects and invoke their inherited and specialized methods.
8. Create hall and bedRoom mixin objects containing additional methods.
9. Create Building and extend it with Area; call super() to initialize the inherited house number.
10. Use Object.assign(Area.prototype, hall, bedRoom) to add mixin methods, create an Area object, and verify the output.

PROGRAM / SOURCE CODE

The following pages contain the submitted JavaScript source code.

```
//Single Inheritance      You, last week • changes …
console.log("Single Inheritance:");
You, last week | 1 author (You)
class Animal {
    constructor(name){
        this.name=name;
    }
    eat(){
        console.log(`${this.name} is eating.`);
    }
}
You, last week | 1 author (You)
class Dog extends Animal{
    bark(){
        console.log(`${this.name} is barking.`)
    }
}
const myDog = new Dog("Buddy");
myDog.eat();
myDog.bark();

//Multilevel Inheritance
console.log("Multilevel Inheritance:");
You, last week | 1 author (You)
class Vehicle {
    move(){
        console.log("Moving forward");
    }
}
You, last week | 1 author (You)
class Bike extends Vehicle{
    wheels(){
        console.log("Bike has 2 Wheels");
    }
}
```

```
class Car extends Bike{
    windows(){
        console.log("Car has 4 Windows");
    }
}
const v = new Car();
v.move();
v.wheels();
v.windows();

//Hierarchial Inheritance
console.log("Hierarchial Inheritance:");
You, last week | 1 author (You)
class User{
    constructor(username){
        this.username = username;
    }
    login(){
        console.log(`${this.username} logged in.`);
    }
}
You, last week | 1 author (You)
class Admin extends User{
    deleteUser(){
        console.log(`${this.username} deleted by admin`);
    }
}
You, last week | 1 author (You)
class Customer extends User{
    checkout(){
        console.log(`${this.username} checked out successfully.`);
    }
}
const a = new Admin("Anvitha");
```

```
const c = new Customer("Sam");
a.login();
c.login();
a.deleteUser();
c.checkout();

//Multiple Inheritance
console.log("Multiple Inheritance:");
//Using Mixins
const hall = {
    sofa(){
        console.log("Sofa is placed in Hall.");
    },
    tv(){
        console.log("Tv is placed in Hall.")
    }
};
const bedRoom = {
    bed(){
        console.log("Bed is in bedroom.");
    },
    closet(){
        console.log("Closet is in Bedroom.");
    }
};
You, last week | 1 author (You)
class Building{
    constructor(houseNo){
        this.houseNo = houseNo;
    }
    house(){
        console.log(`Building House No: ${this.houseNo}`);
    }
}
```

```
class Area extends Building{
    constructor(houseNo,area){
        super(houseNo);
        this.area = area;
    }
}
Object.assign(Area.prototype,hall,bedRoom);
const name = new Area("403","Kommadi");
name.house();
name.sofa();
name.bed();
```

CODE EXPLANATION

JavaScript Element / Feature	Purpose
Animal / Dog	Dog extends Animal and reuses the inherited eat() method while adding bark().
Vehicle / Bike / Car	Car inherits from Bike, which inherits from Vehicle, demonstrating multilevel inheritance.
User / Admin / Customer	Admin and Customer both inherit login() from User but provide different specialized methods.
hall / bedRoom	These objects act as mixins containing sofa(), tv(), bed(), and closet() behaviors.
Building / Area	Area extends Building and uses super() to initialize houseNo.
Object.assign()	Copies mixin methods to Area.prototype, allowing Area objects to use those methods.
Template literals	`${...}` inserts object property values into console messages.

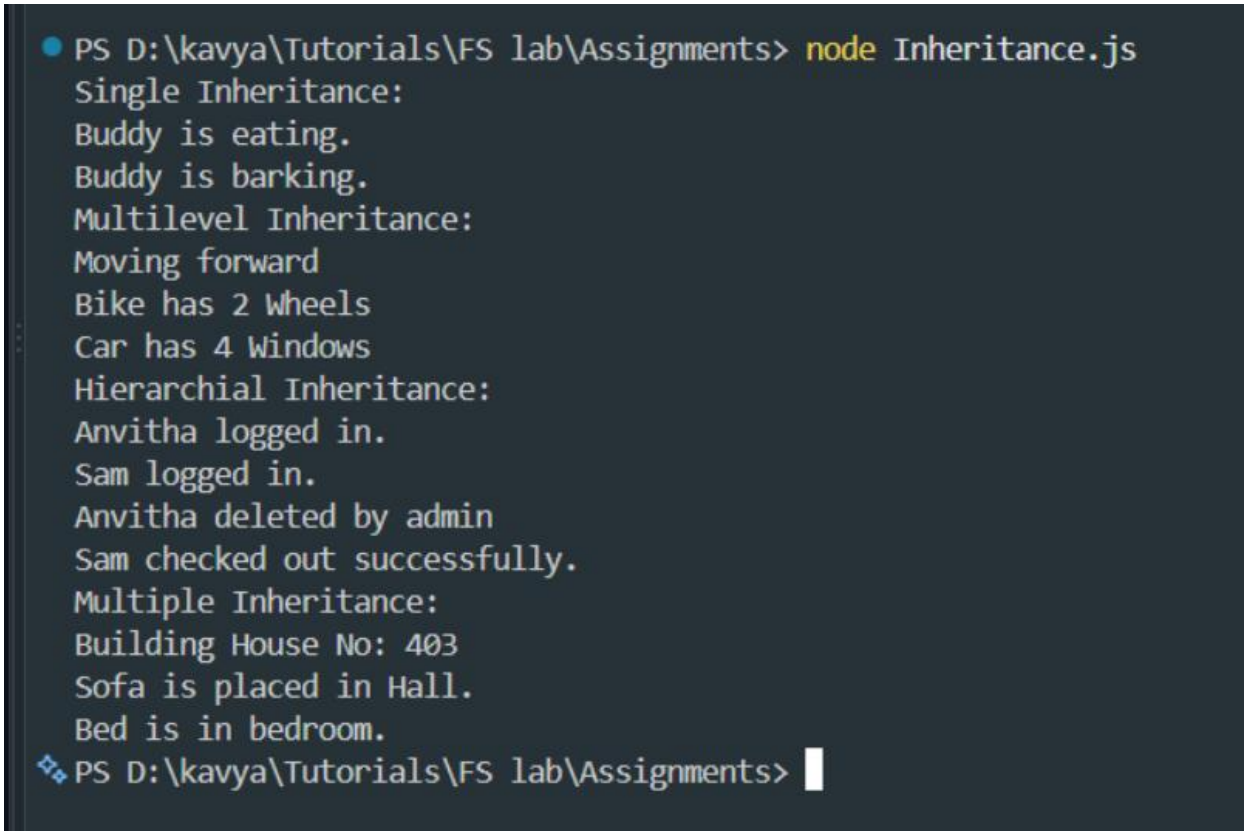
TEST CASES AND EXECUTION DATA

Test Case	Verification	Expected Result	Observed Result / Status
TC-01	Single inheritance	Buddy eating and barking messages are displayed.	Displayed — Pass
TC-02	Multilevel inheritance	Moving, wheel, and window messages are displayed.	Displayed — Pass
TC-03	Hierarchical inheritance	Admin and Customer login/specialized messages are displayed.	Displayed — Pass
TC-04	Multiple inheritance	Building, Hall, and bedroom messages are displayed.	Displayed — Pass

---

OUTPUT

The following is the actual Node.js console output screenshot supplied for this task.



RESULT

Thus, the different types of inheritance were successfully implemented using JavaScript classes and mixins. The program demonstrates single inheritance, multilevel inheritance, hierarchical inheritance, and multiple inheritance behavior using Object.assign(). The supplied console output verifies the execution of all four examples.